Universidade Federal de Santa Maria
Departamento de Computação Aplicada
Prof. Cesar Tadeu Pozzer
Disciplina: Computação Gráfica Avançada
Autores: Flavio Franzin, Cesar Pozzer
12/2024

Compute Shader

1. Introdução

Em computação é extremamente comum realizarmos um determinado processamento a diversos elementos de uma estrutura de dados (arrays, matrizes 2D e 3D). Para acelerar esse tipo de computação é possível fazer o uso de threads em CPU, no entanto, mesmo com esse tipo de paradigma não é possível realizar determinadas tarefas em tempo hábil, uma vez que o hardware não suporta uma quantidade elevada de processamento em paralelo, o que pode aumentar o tempo gasto para o processamento de estruturas maiores.

Em alternativa ao processamento em CPU é possível utilizar as GPU, as quais são dedicadas a processamento paralelo de grandes quantidades de informações. A maneira de processar dados nas GPUs é utilizando Compute Shaders. Em comparação com shaders gráficos, os compute shaders executam fora do pipeline de renderização, e permitem o envio e leitura de dados de acordo com comandos explícitos do programador (na Unity, utilizando códigos em C#). Os dados processados por um compute shader podem ser mantidos na VRAM para uso futuro (inclusive por shaders gráficos) ou retornados a RAM.

Basicamente, compute shaders trabalham de forma semelhante as threads em CPU, uma vez que estes também são orientados a threads. No entanto, mesmo a capacidade de processamento de um core de CPU ser superior ao de uma GPU (considerando 1:1), o número de cores de uma GPU é significantemente superior, o que torna o uso de GPUs propícios às aplicações paralelizáveis e, geralmente, fornecem um significante ganho de desempenho. Isso faz com que as GPUs sejam amplamente visadas para a computação de diversos problemas, gerando uma grande demanda de programadores com conhecimento para trabalhar com essas tecnologias.

Para este estudo de compute shaders iremos utilizar a engine gráfica Unity, no entanto, o paradigma dos compute shaders pode ser empregado em outras engines.

2. Quando usar Compute Shaders

A regra básica para utilização de compute shader é a não (ou mínima) dependência entre dados que serão processados. Assim, podemos destacar um simples comparativo entre processamento paralelizável e não paralelizável.

```
FuncA(Array[] arr)
{
    for(int i = 0; i < arr.length; i++)
    {
        arr[i] = i * i;
    }
}
```

Note que neste exemplo não há dependência entre os elementos do Array, logo este problema torna-se facilmente paralelizável.

```
FuncB(Array[] arr)
{
    for(int i = 1; i < arr.length; i++)
    {
        arr[i] = arr[i - 1] + sqrt(i);
    }
}
```

Por outro lado, neste exemplo há dependência entre os elementos do Array, pois para determinar o valor de uma célula do array é necessário que a célula anterior esteja computada.
Nesse caso, se utilizado compute shaders, o resultado do Array torna-se inconsistente, podendo variar a cada execução.

Outro fator importante é a quantidade de dados. Quando utilizamos compute shaders há uma troca de contexto para que as threads sejam processadas, logo demandando um processamento extra (além da computação que iremos efetuar). Além disso, é necessário considerar o tempo de transferência dos dados entre RAM e VRAM. Por esse motivo, o uso de compute shaders deve ser empregado em grandes quantidades de dados, sendo necessária uma análise minuciosa dos reais ganhos ao utilizar compute shaders.

3. Compute Shader

Quando trabalhamos com compute shader obrigatoriamente estamos vinculados em duas etapas, uma feita em CPU e outra em GPU. Basicamente, o compute shader é processado exclusivamente em GPU, no entanto, a etapa de configuração é feita pela CPU.

3.1 Entendendo Compute Shaders - GPU

Inicialmente, em GPU, devemos criar um compute shader acessando a aba Assets/Create/Shader/computeShader. Por default o Compute Shader já possui seu corpo definido, porém, vamos entender cada segmento de código.

```
#pragma kernel CSMain

RWTexture2D<float4> Result;

[numthreads(8,8,1)]
void CSMain (uint3 id : SV_DispatchThreadID){
	Result[id.xy] = float4(id.x & id.y, (id.x & 15)/15.0, (id.y & 15)/15.0, 0.0);
}
```

A primeira informação que temos é a definição de kernel. Definimos como um kernel toda função que será chamada pela CPU. Dentro de um Compute Shader podemos ter diversos kernels, sendo necessário a definição através da diretiva *#pragma kernel kernel_name*. Funções invocadas a partir de um kernel não se configuram como um, logo não precisam ser explicitadas com a diretiva.

O segundo bloco de código que temos é a definição dos parâmetros, utilizados como inputs/outputs ou variáveis para o compute shader. No código acima notamos o uso de uma textura (Result), que nesse caso será o nosso output. Essa textura, segundo a definição no compute, está habilitada para leitura/escrita (RW) e é composta por 4 canais (float4).

Na sequência, é definido o número de threads que serão utilizadas por cada grupo de threads. A quantidade de grupos é definida através da CPU, a qual abordaremos a fundo na seção 2.3.

Dentro do Kernel é onde será feito o processamento dos dados. Podemos notar no escopo da função o parâmetro id, com a semântica *SV_DispatchThreadID*. Esse parâmetro dá acesso a um ou mais elementos dentro do nosso buffer de saída, nesse caso a textura.

3.2 Entendendo Compute Shaders - CPU

Uma vez que o compute foi definido, podemos dar início a parte da CPU. Para isso devemos criar um arquivo C# que irá invocar a função na GPU.

Para os exemplos a seguir iremos utilizar uma textura, que será passada a GPU para ser processada. Logo, devemos criá-la. Sua definição será uma textura de 16x16 pixels, em formato RGBA (4 canais).

Para visualizar o output gerado pelo compute shader podemos criar uma RawTexture (GameObject->UI->RawImage) e definir o campo *texture (acessando o component RawImage)* com a mesma textura passada ao compute. Após criado, podemos atribuí-lo via inspector.

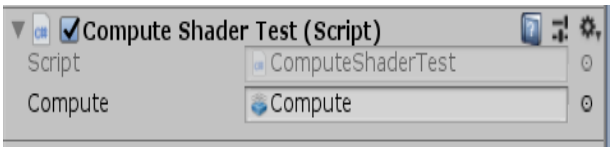
```
static int size = 16;
RenderTexture texGPU;
public RawTexture output; //Associar via inspector (drag and drop) da unity

void Start () {
	//Criação da textura que será passado para a GPU
	texGPU = new RenderTexture(size, size, 1, RenderTextureFormat.ARGB32);
	texGPU.enableRandomWrite = true; //Necessário para que possamos alterar os pixels
	texGPU.filterMode = FilterMode.Point; // Não queremos interpolação entre pixels
	texGPU.useMipMap = false; //Não queremos gerar multi-resolução
	texGPU.Create();

	//Após computar a textura iremos atribui a uma rawImage, para visualizar a saída
	output.texture = texGPU;
}
```

Para executarmos o método no compute shader primeiramente temos que indicar onde o kernel e buffers estão definidos. Nesse caso podemos criar um campo public e atribuir o compute shader via inspector.

```
public ComputeShader compute;
void RunDispatch()
{
    ….
}
```



Como abordamos anteriormente, para que uma função seja chamada pela CPU ela deve ser tachada como kernel, logo, temos que saber qual o kernel iremos invocar. Assim, a primeira etapa é buscar pelo kernel que iremos executar, que nesse caso será o "CSMain". Neste caso o valor retornado deve ser >= 0, caso contrário algo está errado.

```
void RunDispatch()
{
	//Busca pelo kernel (função) que será invocado
	int kernel = compute.FindKernel("CSMain");
	if(kernel < 0)
	    Debug.Log(“Erro”);
}
```

Na sequência, temos que enviar ao compute shader a textura criada anteriormente, pois esta será o nosso buffer de escrita. Se observarmos o código dentro do compute shader veremos a seguinte instrução *“RWTexture2D<float4> Result;”*, que define uma textura com o nome Result. É através desse nome que iremos atribuir a textura criada anteriormente a esse buffer.

Além da textura podemos definir inúmeros outros dados, com diferentes formatos. Obrigatoriamente deve haver uma sincronia entre os tipos de dados enviados da CPU para GPU.

```
void RunDispatch()
{
        int kernel = compute.FindKernel("CSMain");
         if(kernel < 0)
            Debug.Log(“Erro”);
        compute.SetTexture(kernel, "Result", texGPU);
 }
```

Uma vez que o compute foi todo configurado é necessário dimensionar os grupos de threads, que por hora vamos utilizar os valores abaixo e abordar mais a fundo na seção 2.3, Dimensionamento de Thread. Por fim, para invocar a execução do kernel “CSMain” no compute shader é utilizado o método *dispatch,* definido por um kernel e o número de grupos de threads nos eixos X, Y e Z.

Também devemos notar o número de grupos de threads no eixo Z, na qual definimos como 1. Isso ocorre porque usamos uma textura 2D, no entanto, em compute shader podemos definir estruturas de até 3 dimensões. Nos exemplos a seguir iremos trabalhar somente com texturas e buffers de 1D e 2D, logo não comentaremos sobre o eixo Z sendo este sempre definido como 1, mas que segue a mesma lógica do que será comentado na sequência, porém em uma dimensão a mais.

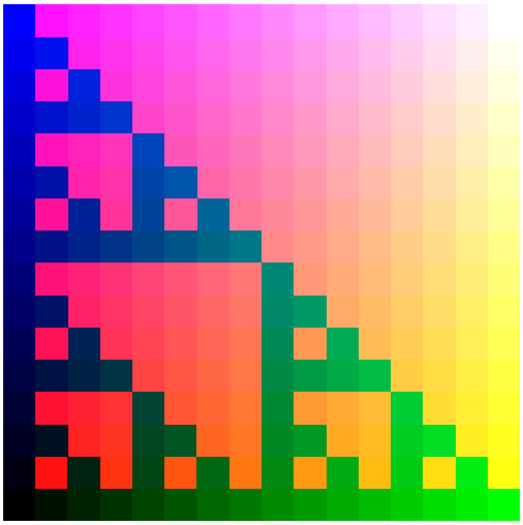
```
void RunDispatch()
{
        ...
        compute.Dispatch(kernel, Mathf.CeilToInt(size / 8f), Mathf.CeilToInt(size / 8f), 1);
 }
```

Até o momento, ao executarmos nossa aplicação já teremos uma saída. Usando o compute shader padrão da Unity o output será algo similar a imagem abaixo. Atente-se a transição de cores entre os pixels, uma vez que a cor é definida através do id definido pela semântica SV_DispatchThreadID. Na seção a seguir iremos esclarecer o que significa esse id e qual a sua funcionalidade.

```
void Start () {
	//Criação da textura que será passado para a GPU
	….

	//Executa a atribuição dos parâmetros na GPU e invoca o método que irá computar
	RunDispatch();

	//Após computar a textura iremos atribui a uma rawImage, para visualizar a saída
	...
}
```



Basicamente o código que invoca o kernel na GPU possui sempre a mesma estrutura.

1. Busca pelo Kernel;
2. Atribuição de parâmetros;
3. Dispatch;
4. Retorno dos dados processados, se houver. (Exemplificado no capítulo 4).

3.3 Compute Shader - Dimensionamento de Thread

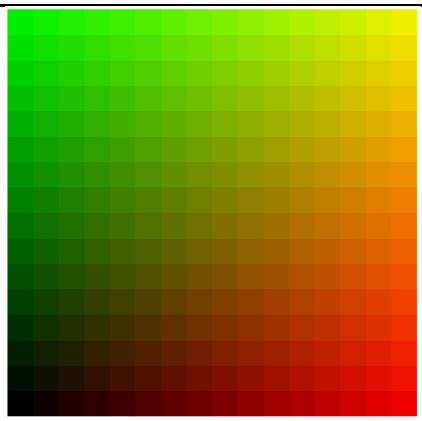
Como abordado anteriormente, o dimensionamento de threads é algo bastante complexo para quem está iniciando com a programação em compute shaders, por esse motivo iremos criar alguns exemplos somente trabalhando com ids das threads.

Quando invocamos o método Dispatch é passado também o número de grupos de threads. No caso acima tínhamos as seguintes configuração:

- Uma textura com 16x16x1 pixels
- O escopo do kernel, no compute shader, está definido com 8x8x1 threads por grupo de threads
- Segundo a nossa definição do dispatch (16/8 x 16/8 x 1/1) nos resultou 4 grupos

Logo, concluímos que temos 4 grupos, com 64 threads cada, resultando em um total de 256 threads para uma textura com 256 pixels. Assim, para esse exemplo, cada pixel será processado por uma thread. Se substituirmos a linha default que define a cor do pixel pelo exemplo abaixo podemos ver os 256 ids através das tonalidades dos pixels.

```
[numthreads(8,8,1)]
void CSMain (uint3 id : SV_DispatchThreadID)
{
    Result[id.xy] = float4(id.x & id.y, (id.x & 15)/15.0, (id.y & 15)/15.0, 0.0);
    Result[id.xy] = float4(id.xy / 16.0, 0.0, 0.0);//dividido por 16 porque esta é a dimensão da textura
}
```

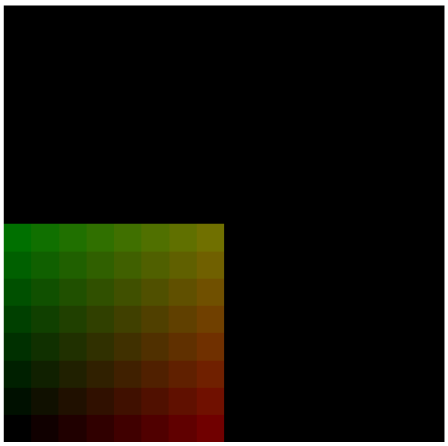


Um segundo teste que faremos é modificar o número de threads por grupo na definição do kernel. Lembrando novamente as definições do dispatch

- uma textura 16x16x1
- com 4 grupos (16/8 x 16/8 x 1/1)
- o escopo do kernel definido com 16 threads por grupo (4x4x1)

Logo, concluímos que temos um total de 64 threads para uma textura de 256 pixels, consequentemente somente ¼ dos pixels serão processados. Isso acontece porque nós causamos um erro no dimensionamento das threads, havendo mais pixels do que threads.

```
[numthreads(4,4,1)]
void CSMain (uint3 id : SV_DispatchThreadID)
{
    Result[id.xy] = float4(id.xy / 16.0, 0.0, 0.0);
}
```



Para tratar esse problema há duas possíveis duas soluções. A primeira solução (A) é redefinir o número de grupos no dispatch,

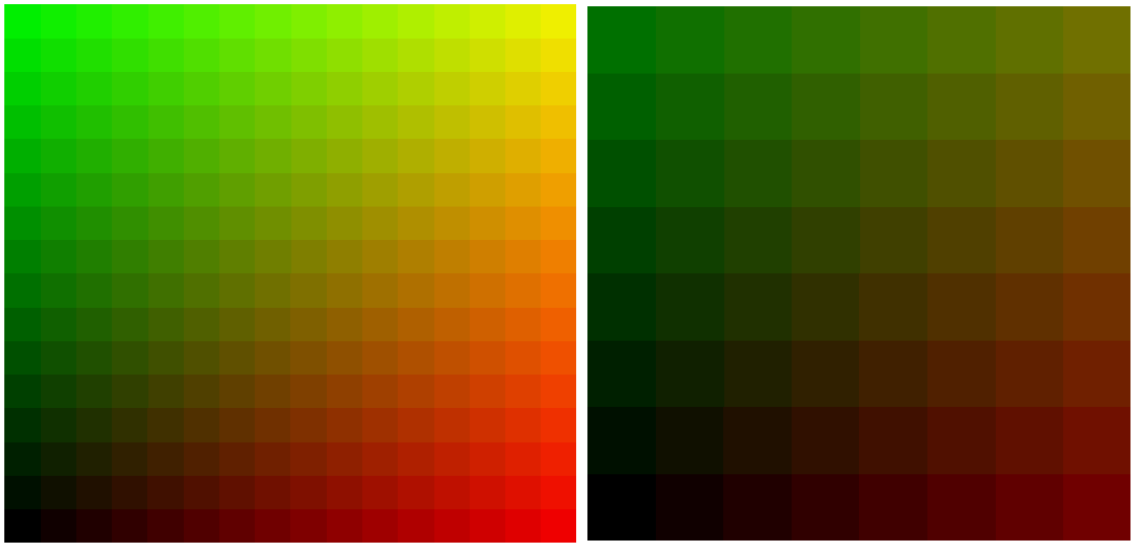
```
void Dispatch(){
	compute.Dispatch(kernel, Mathf.CeilToInt(size / 4f), Mathf.CeilToInt(size / 4f), 1);
}
```

ou (B) fazer com que cada thread processe mais de um pixel, neste caso cada thread deverá processar 2x2 pixels. Como exemplificado no bloco de código a seguir, cada thread irá processar um bloco de 4 pixels (2x2).

```
[numthreads(4,4,1)]
void CSMain (uint3 id : SV_DispatchThreadID){
    int2 init = id * 2;
    int2 size = (id + 1) * 2;

    for (int i = init.x; i < size.x; i++){
        for (int j = init.y; j < size.y; j++) {
            Result[float2(i, j)] = float4(id.xy / 16.0, 0.0, 0.0);// a direita
        }
    }
}
```

Isso fica claro quando olhamos a tonalidade de cada pixel, gerado pelo ID de thread que computou o mesmo. A imagem da esquerda refere-se a solução (A), onde ajustamos o número de grupos. E na direita, solução (B), cada thread processou 4 pixels. Assim, temos proporção de 1:4 tonalidades.



4. Exemplo de Compute Shader - Textura

Para o primeiro exemplo, vamos trabalhar com uma textura. Basicamente iremos definir um degradê a partir da diagonal.

Para isso iremos calcular, para cada pixel, a menor distância entre o pixel atual e todos os elementos da diagonal. Assim, será necessário passarmos a dimensão da nossa textura antes de invocarmos o kernel (para o exemplo em questão consideramos a textura quadrada). Após isso, achamos a menor distância de cada pixel para a diagonal e definimos uma cor com base nessa distância, considerando que a tonalidade maior será quando a distância for igual ou próxima ao valor da diagonal.

```
void RunDispatch()
   {
      ….
      compute.SetTexture(kernel, "Result", texGPU);
      compute.SetFloat("dim", texGPU.width);
      compute.Dispatch(kernel, Mathf.CeilToInt(size / 8), Mathf.CeilToInt(size / 8), 1);
   }
```

```
….
//Variável que recebe a dimensão passada pela CPU -- “compute.SetFloat("dim", texGPU.width);”
float dim;

[numthreads(8,8,1)]
void CSMain(uint3 id : SV_DispatchThreadID)
{
      float smallDis = 99999;

     // calcula a diagonal da textura -- usado para definir uma  cor com base na menor distância.
     // Para isso consideramos a maior distância possível (metade da diagonal) entre um pixel e
     // a diagonal
     float diagonal = sqrt(2 * dim.x * dim.x);

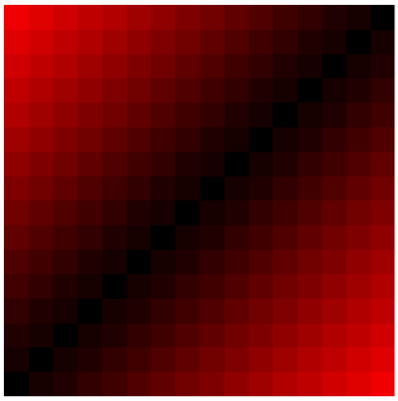
     //Percorre toda a diagonal buscando pelo pixel mais próximo
     for (int i = 0; i < dim; i++)
          smallDis = min(smallDis, distance(id.xy, float2(i, i)));

      //sabemos que a maior distância será a metade da diagonal, logo qualquer distância dividido
      //por esse valor resulta em um valor entre 0-1, nesse caso a nossa tonalidade de vermelho.
     Result[id.xy] = float4(smallDis / (diagonal / 2.0), 0, 0, 1);
}
```

Para o exemplo em questão podemos aplicar uma pequena otimização, considerando que se trata de uma textura quadrada e que a diagonal superior será igual a inferior, é possível calcular somente a diagonal superior e atribuir a ambos os lados.

```
void CSMain(uint3 id : SV_DispatchThreadID){
     if (id.x > id.y)
          return;
     ….

     Result[id.xy] = Result[id.yx] = float4(smallDis / (diagonal / 2.0), 0, 0, 1);
}
```



Como comentamos anteriormente, os compute shaders são empregados para maximizar a performance em determinadas aplicações. Para fins de comparação,

iremos fazer o mesmo procedimento em CPU e realizar uma comparação temporal para dimensionar o quão vantajoso pode ser o uso de compute shader ao se trabalhar com texturas.

Com a ajuda do Profiler podemos calcular os tempos para gerar a textura (1 thread por pixel) com as seguintes configurações: (Hardware utilizado: CPU: Ryzen 5 *** GPU: GTX 1070).

Dimensão	CPU Single thread (ms)	GPU Multi thread (ms)	Ganho de desempenho
64x64	34.51	0.028	1232
128x128	153.52	0.042	3654
256x256	1094.72	0.189	5792
512x512	8611.15	1.190	7176
1024x1024	68980.92	7.421	9295
2048x2048	--	70.72	--
4096x4096	--	216.82	--
8192x8192	--	750.71	--

A tabela acima deixa claro o quão vantajoso é o uso de GPU para o processamento de grandes quantidades de dados. Para o exemplo acima consideramos a CPU single thread, no entanto, se utilizarmos multi thread o desempenho não irá aumentar a ponto de ser superior à GPU.

Abaixo o código utilizado para calcular a textura em CPU.

```
Texture2D texCPU;

void Start () {
	...
	texCPU = new Texture2D(size, size, TextureFormat.ARGB32, false);
	CPUProcess();
}

void CPUProcess() {
	float diagonal = Mathf.Sqrt(2f * texCPU.width * texCPU.width);

	Vector2 pixel = new Vector2();
	Vector2 pixelDiag = new Vector2();
	Color c = new Color();
```

```
        for (int i = 0; i < texCPU.width; i++){
            for (int j = 0; j < texCPU.width; j++){
                if (i > j) continue;

                float min = 99999;

                for (int k = 0; k < texCPU.width; k++) {
                    pixel.x = i;
                    pixel.y = j;
                    pixelDiag.x = pixelDiag.y = k;
                    min = Mathf.Min(min, (pixel - pixelDiag).magnitude);
                }

                c.r = min / (diagonal / 2f);

                texCPU.SetPixel(i, j, c);
                texCPU.SetPixel(j, i, c);
            }
        }

        texCPU.Apply();
}
```

5. Exemplo de Compute Shader - ComputeBuffer

Outra possível aplicação para compute shader é a computação de buffers. Quando computamos texturas estas permanecem em GPU (podem ser recuperadas Link). No entanto, há situações em que desejamos ter os dados computados na memória RAM.

Assim, iremos abordar o uso de compute buffers, que nos permitem computá-los em GPU e retornar os dados a memória principal. Para isso, o primeiro passo é a definição de um compute buffer, definindo o número de elementos e o tamanho do dado utilizado.

```
ComputeBuffer buffer;

void RunDispatch(){
        ….
        buffer = new ComputeBuffer(size, sizeof(float));
        compute.SetBuffer(kernel, "buffer", buffer);
        ...
}
```

A definição do compute buffer também deve ser feita no compute shader.

```
RWStructuredBuffer<float> buffer;
...
void CSMain(uint3 id : SV_DispatchThreadID)
{
    ….
}
```

Também é necessário ajustar as threads, pois anteriormente utilizamos uma estrutura 2D e agora uma 1D.

```
RWStructuredBuffer<float> buffer;

[numthreads(8, 1, 1)]
void CSMain(uint3 id : SV_DispatchThreadID)
{
    ….
}
```

```
ComputeBuffer buffer;

void RunDispatch(){
      …
      //Dimensionamento das  threads. Anteriormente tínhamos 8x8x1 threads por grupo.
      //Agora como temos um Array 1D e queremos um grupo por elemento do buffer
      compute.Dispatch(kernel, bufferSize/8f , 1, 1);
}
```

No exemplo iremos fazer uma computação qualquer, somente para ocupar o hardware.

```
RWStructuredBuffer<float> buffer;

[numthreads(8, 1, 1)]
void CSMain(uint3 id : SV_DispatchThreadID)
{
          buffer[id.x] = lerp(sqrt(pow(cos(id.x) + sin(id.x), 5.0)),
                              sqrt(pow(cos(id.x) + sin(id.x), 3.0)), saturate(cos(id.x)));
}
```

Ao executarmos o compute nada é retornado, pois não definimos a etapa que irá recuperar os dados da VRAM. Com isso, podemos analisar o código abaixo. Nele podemos observar um método *GetData* sendo invocado pelo compute buffer definido acima. Também podemos notar que deve ser passado um array, com as mesmas dimensões utilizadas na definição do compute buffer, como parâmetro do GetData (atente-se também aos overloads). Após a execução do GetData os dados estão na RAM, logo podem ser acessados normalmente através do array *output*.

É importante destacar que ao invocar o método GetData toda a execução da CPU irá esperar o fim do processamento em GPU, logo gerando o travamento na execução da CPU. Esse travamento também pode ocorrer se outro compute shader for invocado para usar os dados gerados pelo nosso exemplo. No entanto, nesse caso, o travamento seria na GPU. O mesmo ocorre quando os dados são utilizados durante a renderização, no entanto, a CPU também é bloqueada, pois o frame de um game só termina após a renderização.

```
ComputeBuffer buffer;

void RunDispatch(){
        ….
        compute.Dispatch(kernel, 1, 1, 1);

      float[] output = new float[size];
      buffer.GetData(output);
}
```

Para compararmos o desempenho entre CPU e GPU iremos utilizar a mesma abordagem do Exemplo 1. Com a mesma lógica definida em CPU (abaixo) e com a ajuda do Profiler podemos mostrar alguns tempos, que nos confirmam a eficiência do uso de compute shaders também para esse tipo de abordagem.

```
void ComputeBufferCPU(){
      float[] output = new float[size];

      for(int i = 0; i < size; i++){
         output[i] = Mathf.Lerp( Mathf.Sqrt(Mathf.Pow(Mathf.Cos((float)i) + Mathf.Sin((float)i), 5f)),
                                 Mathf.Sqrt(Mathf.Pow(Mathf.Cos((float)i) + Mathf.Sin((float)i), 3f)),
                                 Mathf.Clamp01(Mathf.Cos((float)i)));
      }
}
```

Dimensão	CPU Single thread (ms)	GPU Multi thread(ms)		Ganho de desempenho
		Processamento	Total (Proc. + Transferência)	
500	0.58	0.026	0.54	1
5.000	5.31	0.056	1.71	3
50.000	36.19	0.37	2.5	14
500.000	258.60	3.49	6.94	37
5.000.000	2433.51	34.71	49.54	49
50.000.000	24751.29	196.33	249.41	99
500.000.000	--	855.34	1658.68	--

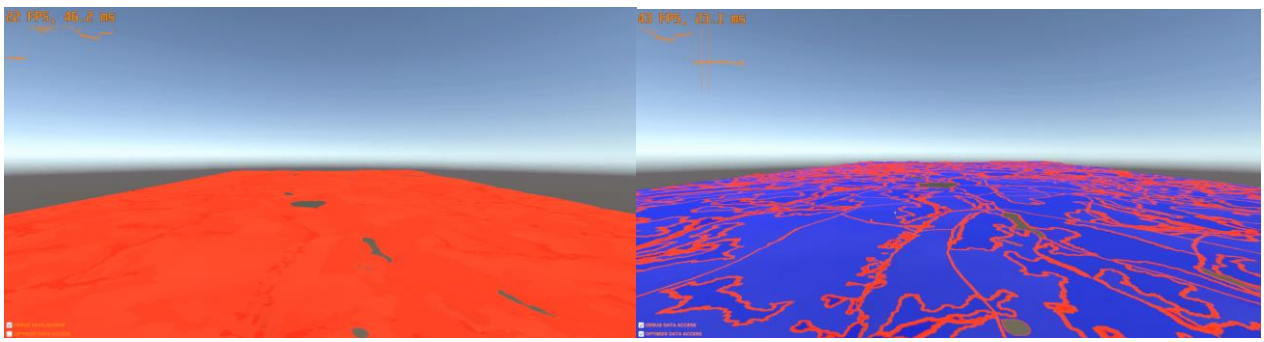
6. Aplicação prática de Compute Shaders

6.1 Geração de mapas para renderização

Durante o refinamento da quadtree, é carregado o HeightMap (HM) dos novos nós gerados. Além disso, para cada nó refinado, dois novos mapas são gerados utilizando compute shaders. Esses mapas são:

1. **NormalMap**: Gerado a partir do HM, este mapa é utilizado nos cálculos de iluminação, fornecendo informações essenciais para a interação da luz com a superfície.
2. **VectorDataMap**: Criado a partir de estruturas predefinidas na GPU que armazenam informações vetoriais como linhas (representando rios, estradas) e polígonos (representando lagos, florestas, ….) necessários para texturização do terreno. Esse mapa realiza a pré-computação dessas informações, com o objetivo de evitar ou reduzir significativamente o acesso constante, durante cada frame de renderização, às estruturas de dados complexas que armazenam esses dados e impactam negativamente a performance.

   Abaixo, os fragmentos em vermelho indicam o acesso às estruturas de dados e os em azul indicam o acesso somente ao VectorDataMap. Observe que na imagem a direita, o VectorDataMap reduz drasticamente o acesso às estruturas com dados raw e aumenta significativamente a performance da renderização.

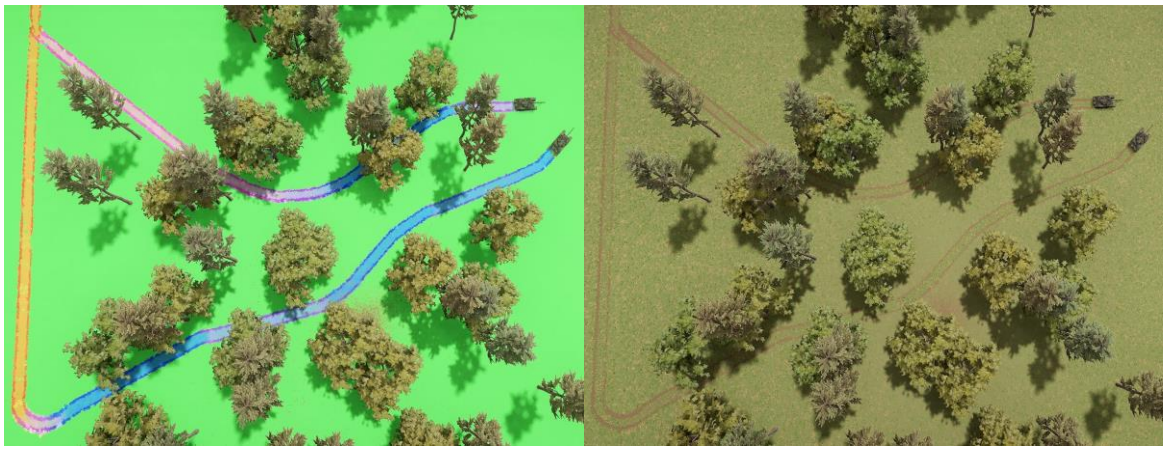


6.2 Deformação de Gramas

Em CPU, é coletado um conjunto de colisores (representados por quads ou esferas), que são enviados à GPU para deformar campos vetoriais descritos como texturas 2D. Esses campos vetoriais são utilizados para definir a orientação de cada *billboard* de grama durante a renderização, simulando o efeito de colisões com veículos e soldados. Assim, dois compute-shaders são utilizados:

- Deformar: baseado em cada colisor (posição, tamanho, orientação e peso), utilizando compute-shaders, é paralelizado a computação das colisões com cada vetor que compõem os campos vetoriais.

- Restauração: Paralelamente, a todo frame, outro *compute shader* realiza a simulação da restauração dos campos vetoriais. Nesse processo, os campos vetoriais deformados são identificados, e um *lerp* é aplicado a cada vetor até que ele retorne à sua forma original, representada por Vector3(0, 1, 0). Nesse cálculo, é considerado o peso aplicado durante a deformação e, assim, conseguimos aplicar deformações permanentes para veículos ou leves movimentos para soldados.



6. Conclusão

Nesse material abordamos conceitos básicos sobre Compute Shader na Unity. Durante o estudo foram abordados diversos exemplos sobre o dimensionamento de threads, uso de texturas e de compute buffers. Juntamente foram elaborados exemplos para comparações entre o custo para processar uma determinada tarefa em CPU e em GPU, deixando clara a eficiência do uso de compute shaders para diversos tipos de problemas.

Neste material foram abordados exemplos simples para demonstrar os potenciais ganhos de performance e a maneira como os compute shaders trabalham. No entanto, compute shaders possuem uma elevada gama de aplicações. No contexto gráfico, podemos destacar a geração procedural de detalhes (ex: erosão e deformação de terrenos), distribuição procedural de elementos gráficos (ex: distribuição de plantas e construções), simulações (ex: partículas, física e fluidos), pós-processamento (ex: anti-aliasing, screen-space-shadow ou blur), entre outros.